A Threat-Driven Approach to Modeling and Verifying Secure Software

Dianxiang Xu
Department of Computer Science
North Dakota State University
Fargo, ND 58105, USA
+1-701-2318185
dianxiang.xu@ndsu.edu

Kendall Nygard
Department of Computer Science
North Dakota State University
Fargo, ND 58105, USA
+1-701-2318203
kendall.nygard@ndsu.edu

ABSTRACT
This paper presents a formal approach to threat-driven modeling and verification of secure software using aspect-oriented Petri nets. Based on the behavior model of intended functions, we identify and build formal models of security threats, which are potential misuses and anomalies of the intended functions that violate security goals. Threat mitigations are further modeled in an aspect-oriented paradigm. Taking Petri nets as a formal basis for modeling behaviors, threats, and mitigations as a whole, we verify properties of and consistency between behaviors and threats, and absence of identified threats from the integrated model of functions and threat mitigations. This makes it possible to achieve a design that is provably resistant to the anticipated threats and thus reduce significant design-level vulnerabilities.

Categories and Subject Descriptors
D.2.4 [**Software Engineering**]: Software and program verification – *formal methods, correctness proofs.*

F.3.1 [**Logics and Meanings of Programs**]: Specifying and Verifying and Reasoning about Programs – *specification techniques, mechanical verification.*

General Terms
Design, Security, Verification.

Keywords
Software security, threat modeling, verification, Petri nets, aspect-oriented software development, aspect-oriented Petri nets.

1. INTRODUCTION
Design-level vulnerabilities are a major source of security risks in software [18]. For example, design-level problems accounted for about 50% of the security flaws uncovered during the Microsoft's "security push" in 2002 [6]. Sufficient protection of software applications from attacks, however, is beyond the capabilities of network and operating system-level security approaches (e.g. cryptography, firewall, and intrusion detection) because they lack knowledge of application semantics. While software engineering principles have suggested treating software security in the early phases of software development [14][2], rigorous methodologies for engineering secure software remain largely open.

This paper presents a formal method for modeling threat-resistant software applications. Different from traditional approaches to achieving security goals, we center round identifying, mitigating, and verifying security threats to the intended functions of a system. Security threats are potential misuses and anomalies that violate security goals. Mitigations are security features for preventing or reducing threats. Threat modeling bridges the gap between (insecure) intended functions and threat mitigations, which are the two main elements of a security design. Security threats are identified and specified according to the intended functions and security goals. Then threats are used to identify threat mitigations. A security design made up of intended functions and threat mitigations is said to be secured from the anticipated threats under two conditions: (1) it is possible for the threats to take place in the intended functions alone – the threats are correct; and (2) the threats will never take place after the mitigations are introduced – the threats are indeed absent.

Our approach consists of three coherent aspects: (1) modeling intended functions, security threats, and threat mitigations as a whole; (2) verifying properties of and consistency between intended functions and security threats; and (3) verifying the absence of security threats from the integrated model of intended functions and threat mitigations. Absence of the security threats implies that the mitigations are indeed effective for securing the system from the anticipated threats. Our approach is based on high level Petri nets - Predicate/Transition (PrT) nets [4][20]. Petri nets are a well-studied formal method with graphical and mathematical notation for distributed systems [13]. Also, Petri nets can represent more complex security threats and variations than attack trees do [12][5]. To better modularize the security concern, we extend Petri nets to support aspect-oriented [8] modeling as security often crosscuts multiple components.

The rest of this paper is organized as follows. Section 2 presents aspect-oriented Petri nets. Section 3 describes threat-driven modeling using aspect-oriented Petri nets. Section 4 describes how to verify correctness of a threat model against the intended behaviors and absence of threats from the mitigated behaviors. Section 5 discusses related work. Section 6 concludes the paper.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.
ASE'05, November 7–11, 2005, Long Beach, California, USA.
Copyright 2005 ACM 1-58113-993-4/05/0011...5.00.

2. ASPECT-ORIENTED PETRI NETS

Following [20], a PrT net consists of predicates (first order places), transitions, arcs, and an initial marking (i.e. state). An arc may be labeled with a tuple of constants or variables. A transition may be associated a logical inscription condition, which must evaluate true to enable the transition. A marking is a distribution of tokens in predicates. Each token is a tuple of constants or variables. A token t residing in predicate p means that proposition p(t) is true. A transition t is enabled under marking M if: (1) for any input predicate of t, there is a token that can be unified with its arc label; and (2) t's inscription condition evaluates true. Firing an enabled transition removes the unified token from each input predicate and adds a token to each output predicate according to the arc labels. This results in a new marking. A sequence of transition firings is called firing sequence.

We enhance Petri nets with the fundamental principles of Aspect-Oriented Programming (AOP, [8]), which modularizes crosscutting concerns into aspects. An aspect-oriented program consists of a number of base modules and aspects that can be woven into an executable whole. In addition to aspects, the basic concepts include joint points, pointcuts, advices, and introduction. A join point is a well-defined point (e.g. method-call) in the flow of a program. A pointcut selects particular join points by filtering out a subset of all join points based on defined criteria. An advice defines additional code to be executed at the join points that match the pointcut specification. An introduction introduces new members to base modules. An aspect encapsulates a group of pointcut, advice, and introduction specifications.

In our approach, an aspect-oriented Petri net model consists of base nets and aspects. As an encapsulated entity of pointcuts, advices, and introductions, each aspect models a separate concern that crosscuts base nets. An introduction, represented by a PrT net (called introduction net), models the functionality common to the advices in the aspect. Joint points can be transitions, predicates, and arcs in base nets. A pointcut selects particular join points based on the defined criteria. For example, *transition pointcut tcut(x): t_2(x), t_3(x)* in Figure 1 defines pointcut *tcut* with a formal parameter *x*, which selects joint points of transitions named t_2 and t_3. An advice for a pointcut specifies the operations to be applied to the joint points that match the pointcut definition. An advice, represented by a PrT net (called advice net), can reference the elements (transitions, predicates, and arcs) in the introduction nets of the same aspect module, which means that two transitions (or predicates) with the same name are equivalent.

Figure 1 is a simple aspect-oriented Petri net model. Figure 1 (a) is the base net. Figure 1(b) is the aspect that defines a pointcut *tcut(x)*, an advice net for *tcut*, and an introduction net. To obtain a whole net model, the aspect can be weaved into the base net as follows: (1) Compose the advice net and the introduction net; (2) Put together the base net and the composed net. Rename transitions and predicatesin the composed introduction and advice net if their names are also found in the base net. In the example, no renaming is necessary. (3) For each transition that matches the definition of pointcut *tcut(x)* (i.e. t_2 and t_3), add an input arc from any input predicate of *tcut* with the same arc label, add an output arc to any output predicate of *tcut* with the same arc label, and combine the transition's inscription formula with that of *tcut*. The variables in arc labels are replaced by the corresponding variables in the transition's formal parameters. For example, t_2's formal parameter is *y* and *tcut*'s parameter is *x*. *x* in the labels of new arcs for t_2 is replaced by <*y*>. (4) Remove pointcut transitions (i.e. *tcut*) and associated arcs in the composed net.

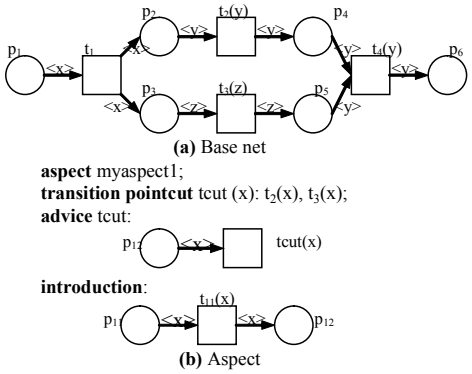


Figure 1. A simple aspect-oriented net model.

Figure 2 shows the woven net for the aspect-oriented model in Figure 1. $p_{12}(y)/p_{12}(z)$ is a new precondition for transition t_2/t_3. Before t_2 or t_3 can be fired, the new precondition must be achieved, e.g. by firing t_{11}. So the advice for *pointcut tcut(x)* is in essence an advice of the *before* type in AOP. Due to limited space, this paper will not discuss predicate or arc pointcuts.

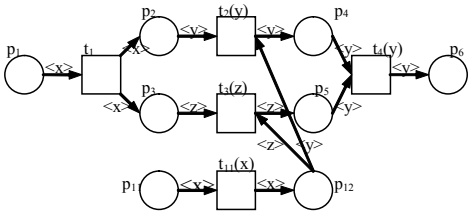


Figure 2. Woven net for the aspect-oriented model in Figure 1

3. THREAT-DRIVEN MODELING

3.1 Building a Secure Software Model

The general process of building a threat-driven security model is as follows: (1) defining the behavior model of intended functions according to functional requirements; (2) constructing the threat model by formalizing each potential misuse and anomaly of the intended behaviors that violate any of the security goals; and (3) specifying the mitigations for misuses or anomalies in the threat model. This process can be done iteratively, and security models can be built at different levels of abstractions.

Definition 1: A security model is a 4-tuple <ϕ, α, τ, β>, where ϕ is the set of system properties or goals, α is the model of intended functions, τ is the threat model, and β is the mitigation model.

Definition 1 is independent of the concrete representations of ϕ, α, τ, and β. It reflects the general adversary's perspective of software security. A specific representation, however, should support formal or informal verification. This paper uses PrT nets to specify intended functions α and threats τ, and uses PrT net based aspects to model mitigations β. Hence, α and β form an aspect-oriented PrT net model. The PrT net obtained by weaving β into α is called the mitigated behavior model. Each property or goal in ϕ can be represented either formally or informally.

Building a behavior model of the intended functions of a system may follow traditional Petri net methods for system modeling. To illustrate our approach, we use a simplified shopping cart application, which allows customers to purchase products online. A shopping cart application consists of several components [11], including session manager, product category manager, payment gateway, and back-end database manager. These components may be provided by different parties. Many shopping cart systems failed because of security vulnerabilities (e.g., customers may purchase products at a reduced price) [11]. Consider "placing an order" in the shopping cart application. After downloading the shopping webpage, a customer may browse the product category generated by the category component and select a product, add the selected product into the shopping cart, delete an item from the cart, check out, and make payment. Before adding an item into the cart, the customer may modify the default quantity of the product. Figure 3 shows the PrT net model of "placing an order". For clarity, most of the arc labels are omitted. Transition *Browse(u, c, s, p, n)* means customer *u* browses and selects a product. The code, specification, price, and quantity of the selected product are *c*, *s*, *p*, and n (1 is the default), respectively. *EditItem(u, c, s, p_1, n_1, p_2, n_2)* means customer *u* decides to buy n_2 items of *c* at price p_2 *other than* n_1 items at price p_1. After the customer requests to check out, the storefront confirms the order by generating an order number for the customer.

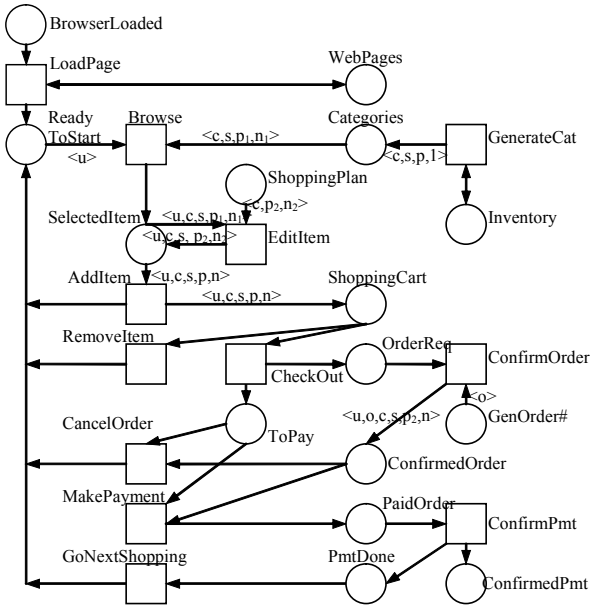


Figure 3. The behavior model of placing an order

3.2 Threat Modeling

Definition 2. Given PrT net α, PrT net τ, and system goals ϕ. Let τ_T and α_T denote the transition sets of τ and α, respectively. τ is a threat model w.r.t. α and ϕ if: (1) $\tau_T \cap \alpha_T \neq \emptyset$; and (2) For any $t \in \tau_T$ and $t \notin \alpha_T$, firing of *t* violates some security goal $g \in \phi$.

A transition *t* in the PrT net of a threat model is either a security threat or a transition in the PrT net of the corresponding behavior model. If transition *t* occurs in both nets of α and τ, then *t* in α and *t* in τ refer to the same action or event. Such common transitions bind the two models together. If transition *t* in threat model τ does not appear in the behavior model α, then *t* is a security threat, whose firing indicates that an attack has happened. Generally, a threat model specifies all identified threats for a given behavior model and security goals. Predicates in a threat model represent ordering and causal relationships among attack steps. To build a threat model, we may check transitions (i.e. actions) in the behavior model α to determine if they are possibly associated with misuses or anomalies that violate security goals. In Figure 3, a potential threat to the security goal of integrity is that a customer may purchase products at a reduced price. This type of vulnerabilities was exploited in several shopping cart systems, for example Cart32 [11]. They used hidden form fields within the html source code that contain product information such as price, weight, quantity, and identification. A malicious customer could save the webpage of a selected item, change the price of the item, and then submit the order request. Here client-side input (i.e. quantity of a selected item) is desirable. Figure 4 shows the PrT net model of the integrity threat, where *u*, *o*, *c*, *s*, *p*, and *n* represent customer, order#, product code, product specification, price, and quantity, respectively. The attack happens when the price p_2 of product *c* of a confirmed order *o* by customer *u* is less than (or more generally different from) the stock price p_1.

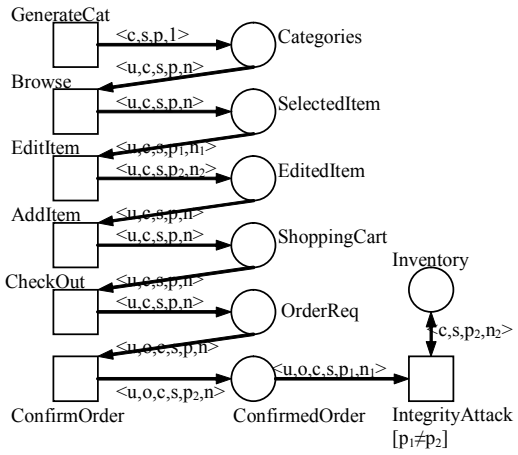
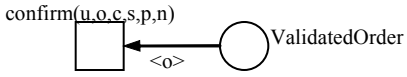


Figure 4. An integrity threat

3.3 Mitigation Modeling

For each threat in a threat model, we can define one or more mitigation aspect by applying certain assurance techniques for countermeasure. One security solution to the above integrity threat is to make the server side validate the product price before confirming an order. We define it as an *OrderValidation* aspect, which validates the price submitted by a customer (i.e. predicate *OrderReq*) against the stock price (predicate *Inventory*). The price must be validated before the system can confirm an order. A new precondition (i.e. the order being validated) is therefore imposed on transition *ConfirmOrder* in the original behavior model. This is achieved by including *ConfirmOrder* in a transition pointcut with a *before* advice. Figure 5 shows the aspect specification. For the shopping cart application, we have also modeled threats to other security goals such as privacy and confidentiality when elaborating on such functions as canceling an order and tracking order status. For example, a customer may track status of or even cancel orders placed by someone else.

aspect OrderValidation // mitigate the integrity threat
transition pointcut confirm(u,o,c,s,p,n):
ConfirmOrder(u,o,c,s,p,n)
predicate pointcut pricecut(u,c,s,p,n): Inventory
predicate pointcut ordercut(u,c,s,p,n): OrderReq
advice confirm



advice pricecut
fuse with Price
advice ordercut
fuse with Order

introduction

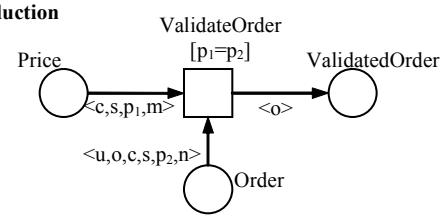


Figure 5. The OrderValidation aspect

4. VERIFYING SECURITY MODELS

Although a security model can be analyzed from several different perspectives, this paper focuses on verifying correctness of a threat model and absence of the threats from the mitigated behavior model.

4.1 Verifying Threat Models

Definition 3: Given a security model <ϕ, α, τ, β>. The threat model τ is sound with respect to the behavior model α only if, for any threat *t* in τ, it is possible for *t* to take place in α.

If there is a threat in the threat model that never happens in the behavior model, then either the behavior model or the threat model is problematic. The process for verifying a threat in the threat model consists of two main steps:

(1) Constructing a search tree for the threat according to the threat model.

The transition representing the threat is the root of the tree. Each path from a leaf node to the root is a possible way to make the threat happen. The tree is constructed recursively by starting with the threat transition and reasoning about its preconditions, i.e. which transitions can produce its preconditions. Each of these transitions either represents another threat or has a corresponding transition in the behavior model. A transition is a leaf if it has no precondition or its preconditions are not connected to other transitions. Each path maintains a set of constraints derived from the transition inscriptions.

(2) Verifying if each path in the search tree can be (part of) a firing sequence in the behavior model.

If there is no such path, the threat will never take place, and thus the threat model is not sound with respect to the behavior model. For a threat with multiple paths, each path is actually intended to be a way to reach the attack from the modeler's perspective. If there exists a path that can never be a firing sequence in the behavior model, then the threat model is likely incorrect. Therefore, we further strengthen the previous definition.

Definition 4: Given a model <ϕ, α, τ, β>. The threat model τ is correct with respect to the behavior model α if, for any threat *t* in τ, it is possible for every path of *t* to take place in α.

It is not difficult to show that the threat in Figure 4 is correct with respect to the behavior model in Figure 3.

4.2 Verifying Absence of Threats

Definition 5: Given a model <ϕ, α, τ, β>. <ϕ, α, β> is a secure design with respect to τ if, for any threat *t* in τ, it is impossible for *t* to take place in the mitigated behavior model of α and β.

Recall that the mitigated behavior model is obtained by weaving β into α. For example, the woven net for the base net in Figure 3 and the aspect in Figure 5 is shown in Figure 6, where the dashed box, circle, and arrows are the transition, predicate, and arcs introduced by the aspect.

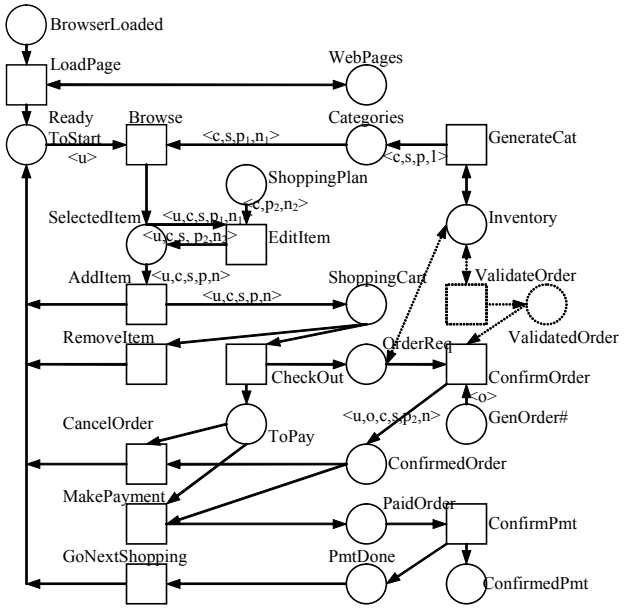


Figure 6. The mitigated model for the shopping system

Suppose α and τ have proven correct. If <ϕ, α, β> is insecure, it is likely because the mitigation aspect is ineffective. In this case, more work on the mitigation needs to be done. Similar to verifying the correctness of a threat model, we can verify the absence of threats in the mitigated behavior model.

Again consider the threat example in Figure 3. It is not difficult to show that the mitigated model in Figure 6 is free from the integrity threat because transition *ConfirmOrder(u,c,s,p_1,n_1,o)* in the only threat path will never fire when *Inventory(c, p_2, n_2)* and $p_1=p_2$. Therefore, transition *IntegrityAttack* in the threat model will never fire. The fact that <ϕ, α, β> is proven resistant to τ does not mean it is absolutely secure because a mitigation aspect (e.g. SSL) may be subject to security threats. The design is insecure if the mitigation can be compromised. When applying mitigations, we should be aware of and document the underlying constraints and assumptions.

5. RELATED WORK

Threat modeling [17][7] is a structured approach to securing software design. It first determines threats to the system based on application decomposition, ranks them by decreasing risk, and then choose techniques to mitigate them. The suggested tools, data flow diagrams and attack trees [16], do not support verification of threat models. In contrast, formal methods can play a critical role in achieving a high degree of reliability and trustworthiness [14][19]. Although modeling and verifying protocol security has long been a focus, formal methods can go far beyond [19]. Deng et al. have proposed a formal approach to modeling and analyzing security system architectures [1]. The approach uses Petri nets to model behaviors of architectural components and connectors and temporal logic to specify constraints on and security properties of system behaviors. Thus security properties about role-based access control can be verified against the behavior model. Lotz has proposed a process-algebra based formal approach for developing secure communication systems with the aid of threat scenarios [10]. In comparison, our approach specifies threats according to misuses and anomalies of the intended behaviors and handles security features for mitigating the threats in the aspect-oriented paradigm.

To model secure software at the design-level, security-oriented [9] and aspect-oriented extensions [15][3] to UML are emerging. Since software security is in nature a crosscutting concern, Aspect-Oriented Software Development as a structured approach to addressing security concerns has gained increasing attention. Ray et al. treat role-based access control mechanisms for secure software as aspects and use extended UML notation to model these crosscutting concerns [15]. Gao et al. have conducted a case study on using aspect-oriented UML to model the role-based access control in a CORBA-based distributed system [3], where aspect-orientation provided a means for dealing with evolution of security policies. The above approach does not support rigorous analysis due to its informal nature. Security is not addressed from the perspective of threats.

6. CONCLUSIONS

The formal threat-driven modeling and verification of secure software presented in this paper has three unique aspects: (1) a uniform method for modeling system behaviors, threats, and mitigations as a whole; (2) a formal method for achieving secure design by verifying correctness of threat models and absence of threats; and (3) a formal treatment of aspect-orientation for modeling and reasoning about crosscutting threat mitigations.

Our approach supports reasoning about partial specification (e.g. the example in this paper is partially specified) and design refinement at different levels of abstractions. We can decompose intended functions, security threats, and threat mitigations. Decomposition of intended functions often entails decomposition of threats and mitigations because they need to remain at the same level of abstraction. When intended functions are being decomposed, more details about threats and mitigations will be revealed. In addition, the approach can be applied as a lightweight formal complement to the informal threat modeling [17][7]. To secure the design of a complex system, for instance, the formal technique may concentrate on threats with high security risks in critical components of the system.

7. ACKNOWLEDGEMENTS

This work was supported in part by the NSF under grant EPS-0132289.

8. REFERENCES

[1] Deng, Y., Wang, J., Tsai, J., and Beznosov, K. An approach for modeling and analysis of security system architectures. *IEEE Trans. on Knowledge and Data Engineering*. 15, 5, 2003, 1099-1119.

[2] Devanbu, P.T. and Stubblebine. Software engineering for security: A roadmap. In *The Future of Software Engineering*, 2000, 227-239.

[3] Gao, S., Deng, Y., Yu, H., He, X., Beznosov, K., and Cooper, K. Applying aspect-orientation in designing security systems: A case study. In *Proc. of SEKE'04*, 2004.

[4] Genrich, H.J. Predicate/transition nets. *Petri Nets: Central Models and Their Properties*, W. Brauer, W. Resig, and G. Rozenberg (eds.), LNCS 254, 1987, 207-247.

[5] Ho,Y., Frincke,D., and Tobin, D. Jr. Planning, Petri nets, and intrusion detection. In *Proc. of the 21st National Information Systems Security Conference (NISSC'98)*, 1998.

[6] Hoglund, G. and McGraw, G. *Exploiting Software: How to Break Code*. Addison-Wesley, 2004.

[7] Howard, M. and LeBlanc, D. *Writing Secure Code*. Microsoft Press. 2nd edition, 2003.

[8] Kiczales, G., Lamping, J., Mendhekar, A., Maeda, C., Lopes, C.V., Loingtier, J.M., and Irwin, J. Aspect oriented programming. In *Proc. of ECOOP'97*, 1997, 220-242.

[9] Lodderstedt, T., Basin, D., and Doser, Jurgen. SecureUML: A UML-based modeling language for model-driven security. In *Proc. of UML'02*, 2002.

[10] Lotz, V. Threat scenarios as a means to formally develop secure systems. *Journal of Computer Security* 5,1997, 31-67.

[11] McClure, S., Shah, S., and Shah, S. *Web Hacking: Attacks and Defense*. Addison-Wesley, 2003.

[12] McDermott, J. Attack net penetration testing. In *Proc. of the 2000 Workshop on New Security Paradigms*, 2000, 15-21.

[13] Murata, T. Petri nets: Properties, analysis and applications, *Proc. of the IEEE*, 77, 4, 1989, 541-580.

[14] Neumann, P.G. Principled assuredly trustworthy composable architectures, Project Report, Computer Science Laboratory. SRI International, 2004.

[15] Ray, I., France, R., Li, N. and Georg, G. An aspect-based approach to modeling access control concerns. *Information and Software Technology*, 46, 9, 2004, 575-587.

[16] Schneier, B. Attack trees. *Dr. Dobb's Journal of Software Tools* 24, 12, 1999, 21-29.

[17] Swiderski, F. and Snyder, W. *Threat Modeling*. Microsoft Press, 2004.

[18] Viega, J. and McGraw, G. *Building Secure Software: How to Avoid Security Problems in the Right Way*. Addison Wesley, 2002.

[19] Wing, J. A symbiotic relationship between formal methods and security, In *Proc. NSF Workshop on Computer Security, Fault Tolerance, and Software Assurance: From Needs to Solution*, 1998.

[20] Xu, D., Yin, J., Deng, Y. and Ding, J. A formal architectural model for logical agent mobility, *IEEE Trans. on Software Engineering*, 29, 1, 2003, 31-45.